Tabla de contenidos

1. Variables de clases y de instancia
2. Encapsulamiento
    a. Encapsulamiento e interfaces
    b. Encapsulamiento en Python
3. Aprender más

Variables de clases y de instancia

Muchas veces, en OOP, necesitamos tener un valor que sea compartido por todas las instancias de una clase, pero que esta a su vez sea una variable propia de la clase. Para esto, podemos definir una variable de clase, simplemente declarándola dentro de su definición. Luego, para utilizarla basta llamarla como si fuera un atributo de la clase. Supongamos que nos piden implementar el sistema registro de personas en el registro civil. Sabemos que todas las personas tienen un RUN (o RUT), y que este es único (dos personas no pueden tener un mismo RUN). Es claro que podríamos implementar una clase `Persona` que tenga un atributo `run`, pero ¿cómo nos aseguramos que todas las personas tengan un `run` único? En términos de programación, ¿cómo podemos hacer que cada instancia de `Persona` tenga un `run` único? Para esto podemos utilizar una variable de clase. A continuación, se muestra un ejemplo de cómo podríamos implementar esto en Python, utilizando variables de clase.

```
class Persona:
    run = 1

    def __init__(self, nombre):
        self.nombre = nombre
        self.run = Persona.run
        Persona.run += 1

    def presentarse(self):
        print(f'Hola, soy {self.nombre} y mi run es {self.run}')

p1 = Persona('Pablo')
p2 = Persona('Dani')
p3 = Persona('Fran')
p4 = Persona('Lucas')
p5 = Persona('Paqui')

p1.presentarse()
p2.presentarse()
p3.presentarse()
p4.presentarse()
p5.presentarse()
```

```
Hola, soy Pablo y mi run es 1
Hola, soy Dani y mi run es 2
```

```
Hola, soy Fran y mi run es 3
Hola, soy Lucas y mi run es 4
Hola, soy Paqui y mi run es 5
```

Ahora, cada persona que se registre tendrá un run único. El run de cada instancia pasa a ser propio de la instancia, y el valor del siguiente run quedará guardado en la variable de clase.

Encapsulamiento

Una característica muy favorecida en OOP es el **encapsulamiento**. El encapsulamiento se refiere al ocultamiento de los atributos de un objeto de manera que éstos sólo puedan ser modificados mediante los métodos que el programador defina. Esto evita que un objeto que interactúa con otro pueda observar o modificar elementos que sean internos al funcionamiento del objeto, y obliga a que toda la interacción sea de la manera que el programador la definió, contribuyendo a reducir la ocurrencia de errores.

Usemos un objeto clase `Auto` para ejemplificar este encapsulamiento:

> Un `Auto` tiene atributos `color` y `modelo` que son interesantes para un objeto de la clase `Persona` que quiera interactuar con él. Sin embargo, también puede haber atributos como `disco_de_embrague`, o `palanca_de_cambios`, que también son atributos de `Auto`, pero son internos a la construcción y al funcionamiento de un objeto de clase `Auto`. Otros objetos de clase `Auto`, o de otras clases como `Persona` no necesitan interactuar con ellos, o al menos no directamente. Los atributos `disco_de_embrague` y `palanca_de_cambios` están **encapsulados** dentro de la clase `Auto`.

El encapsulamiento nos ayudará a alcanzar un mejor nivel de abstracción en el modelamiento de nuestros programas al definir qué atributos de un objeto son de interés para otros objetos y cuáles atributos son de interés únicamente para el comportamiento interno del objeto y, por lo tanto, deberían permanecer ocultos o *encapsulados* dentro del objeto. Veremos que un correcto uso del encapsulamiento de atributos lleva a un código más limpio.

Encapsulamiento e interfaces

En programación, una interfaz es una *fachada* para proteger la implementación de una clase e interactuar con otros objetos. La interfaz define el **conjunto de atributos y métodos** de un objeto que son *expuestos* u ofrecidos por la clase para poder interactuar con otros objetos.

Utilizan el ejemplo de la clase `Auto`:

> Una interfaz que puede ser ofrecida a un objeto de clase `Conductor` puede incluir atributos como `marca` y `modelo` y `año`, y métodos como `conducir`, `vender`, ó `pintar`. No queremos que un objeto de clase `Conductor` pueda modificar directamente el atributo `kilometraje`, sino que sólo lo puede hacer utilizando el método `conducir`. Por otro lado, si consideramos la interacción con un objeto de clase `Mecánico`, podríamos pensar en una interfaz con atributos como `nivel_de_aceite`, y métodos como `abrir_capot`, `cambiar_vidrios` o `cambiar_aceite`.

El nivel de detalle de la interfaz se denomina **abstracción**. En el siguiente ejemplo, todos los atributos `marca`, `año`, `kilometraje` y los métodos `conducir`, `pintar`, `vender` siguen siendo parte de la clase `Auto`. Sin embargo, hemos definido una **interfaz** con cierto nivel de abstracción para interactuar con la clase `Conductor`, ocultando o abstrayendo al `Conductor` de detalles internos del `Auto`, por ejemplo, esa interfaz no incluye el atributo `kilometraje` para que así el `Conductor` no lo modifique.

Encapsulamiento en Python

En lenguajes tradicionales que usan OOP, como por ejemplo Java y C#, es posible definir atributos o métodos que pueden ser accedidos desde fuera del objeto (*públicos*), y otros que sólo pueden ser utilizados internamente (*privados*). Esta característica es usada para implementar **encapsulamiento**. Al declarar algunos atributos o métodos como *privados*, estos prohibiendo que puedan ser invocados desde código externo a la clase, y el lenguaje evita que el programador viole estas reglas.

En Python esta diferencia no existe, y **todos los atributos y métodos de un objeto son públicos**. Esto complicaría la implementación del encapsulamiento en Python. Sin embargo, existe una convención que permite *sugerir* que un método o atributo es de uso únicamente interno. Esto se hace agregando un caracter *underscore* (`_`) al inicio del atributo o método, como en el siguiente ejemplo:

```
class Televisor:
    ''' Clase que modela un televisor.
    '''

    def __init__(self, pulgadas, marca):
        self.pulgadas = pulgadas
        self.marca = marca
        self.encendido = False
        self.canal_actual = 0
        self._clave = "tv123"

    def encender(self):
        self.encendido = True

    def apagar(self):
        self.encendido = False

    def cambiar_canal(self, nuevo_canal):
        self._decodificar_imagen()
        self.canal_actual = nuevo_canal

    def _decodificar_imagen(self):
        print("Estoy convirtiendo una señal eléctrica en la imagen que
estás viendo.")
```

En este ejemplo, podemos notar que la clase `Televisor` tiene los métodos `encender`, `apagar`, `cambiar_canal` y `_decodificar_imagen`. Digamos que queremos crear objetos de la clase `Televisor`.

```
televisor1 = Televisor(17, 'zony')
televisor2 = Televisor(21, 'zamsung')
```

Estos televisores que hemos creado deberían poder ser encendidos y apagados. También deberíamos poder cambiar el canal. Pero no necesitamos decirle al televisor que decodifique la imagen. Ésta es una operación que se realiza automáticamente, cada vez que se cambia el canal. Como el método `_decodificar_imagen` empieza con *underscore*, **por convención** éste no debe ser llamado fuera de la clase. Lo mismo ocurre con el atributo `_clave`, que es un parámetro interno del televisor. Sin embargo, como todo esto sólo una convención, **aún podemos acceder a ellos directamente**.

```
televisor1._decodificar_imagen()      ## Esto no debería hacerlo, pero lo estoy haciendo igual :(
print(f"No debería poder leer que la clave es {televisor1._clave}")
```

```
Estoy convirtiendo una señal eléctrica en la imagen que estás viendo.
No debería poder leer que la clave es tv123
```

(Bonus) Atributos Privados en Python

Si bien fijamos una convención para definir atributos privados anteponiéndole un guión bajo, pareciera que esta convención no hace realmente ninguna diferencia para el intérprete de Python (aun que sí para nuestra propia comprensión del código). Si queremos (casi) realmente tener atributos y métodos que no puedan ser llamados directamente, podemos iniciar con *double underscore* como en el siguiente ejemplo.

```
class Televisor:
    ''' Clase que modela un televisor.
    '''

    def __init__(self, pulgadas, marca):
        self.pulgadas = pulgadas
        self.marca = marca
        self.encendido = False
        self.canal_actual = 0
        self._clave = "tv123"
        self.__clavesecreta = "tv456"

    def encender(self):
        self.encendido = True

    def apagar(self):
        self.encendido = False

    def cambiar_canal(self, nuevo_canal):
```

```
        self._decodificar_imagen()
        self.canal_actual = nuevo_canal

    def _decodificar_imagen(self):
        print("Estoy convirtiendo una señal eléctrica en la imagen que
estás viendo.")

    def __mostrar_canal_prohibido(self):
        print("Esto permite ver el canal del curling, pero usted no
debe saberlo.")

televisor1 = Televisor(17, 'zony')
televisor2 = Televisor(21, 'zamsung')
```

Y entonces si queremos acceder a estos elementos:

```
print(televisor1.__clavesecreta)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call
last)
Cell In[6], line 1
----> 1 print(televisor1.__clavesecreta)

AttributeError: 'Televisor' object has no attribute '__clavesecreta'
```

```
televisor1.__mostrar_canal_prohibido()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call
last)
Cell In[7], line 1
----> 1 televisor1.__mostrar_canal_prohibido()

AttributeError: 'Televisor' object has no attribute
'__mostrar_canal_prohibido'
```

Podemos ver que, a pesar que los atributos existen, Python pareciera ser incapaz de encontrarlos y nos arroja un error. La verdad es que todo esto es un **truco** de la implementación de Python (y Python tiene muchos) para proveer algo similar a los atributos y métodos privados. Cuando un atributo o método empieza con *doble underscore*, Python reemplaza internamente sus nombres por `_NombreDeLaClase__atributo_o_metodo_secreto`, y por lo tanto podemos ser más astutos y escribir:

```
televisor1._Televisor__mostrar_canal_prohibido()
print(f"Ahora sí puedo ver que la clave secreta es
{televisor1._Televisor__clavesecreta}")
```

```
Esto permite ver el canal del curling, pero usted no debe saberlo.
Ahora sí puedo ver que la clave secreta es tv456
```

Este truco se conoce como *name mangling*. No ocurre, sin embargo, cuando el nombre del método termina también con *doble underscore*, por lo cual sí podemos llamar directamente métodos como `televisor1.__str__()`. Estas características son, en cualquier caso, exclusivas de Python y su objetivo es disminuir la posibilidad de errores por parte del programador al proveer algo que simula la existencia de atributos y métodos privados en un lenguaje que por diseño no los tiene.